Detailed logic of how you implemented each virtual memory function

- **set_physical_mem():** Set up bitmaps for both physical and virtual memory, as well as malloc for both virtual memory and physical memory. Set an initialized variable to 1 to prevent other threads from calling set_physical_mem multiple times and resetting everything.
    - We state that our page directory is the 0th page in physical memory.
- **translate():** extract various parts of the physical memory. Walks the page directory. Adds the value stored at the requested location to the start of the virtual memory and returns that.
- **map_page():** Searches the pagetable directory for an existing page table that matches the virtual address's respective page number. If it does not exist, create one by finding the next free physical memory block using an altered get_next_avail for physical memory. Once the directory's link to pagetable level 2 exists, connect the level 2 pagetable location to the physical address.
- **get_next_avail():**
- **n_malloc():** Searches the virtual bitmap and returns the virtual address that corresponds to the first of the needed consecutive bits, while also establishing a mapping with each page that is mapped.
- **n_free():** Similar to malloc, the corresponding virtual pages are marked as free, and their respective physical pages are marked as free
- **put_data():** First translates the virtual address into a physical address. If the physical address is not allocated, find the next free bit and set it to used. Then, copy the data into the physical address.
- **get_data():** First translates the virtual address into a physical address. If the physical address is not allocated, just return since it means that the virtual address is invalid. If there is a physical address, copy the physical address into the user buffer.

Benchmark output for Part 1 and the observed TLB miss rate in Part 2.

- In both cases (mtest and test), executing the matrix multiplication test would give us

    5 5 5 5 5
    5 5 5 5 5
    5 5 5 5 5
    5 5 5 5 5
    5 5 5 5 5
- As long as x = 1, This same matrix would remain consistent even with changes to the page size, and the amount of threads tested in mtest.

Support for Different Page Sizes

- We tested on 4 KB, 8 KB, and 64 KB. In all instances, the pages that were accessed were consistent among all page sizes, and made logical sense in both the single threaded and multi-threaded versions. 4 KB works consistently with up to 50 threads, while 8 KB and 64 KB seem to work with 15.

Possible Issues

- The TLB has not been created.